

Project Report: Retrieval-Augmented Generation (RAG) System for Insurance Domain using LangChain

1. Introduction

This project demonstrates the design and development of a Retrieval-Augmented Generation (RAG) system for the insurance domain using LangChain. The objective is to enable efficient, accurate, and context-aware question answering over large sets of insurance policy documents.

2. Objectives

1. Develop a question-answering system capable of interpreting and responding to user queries using policy documents.
 2. Use embeddings and LLMs to understand and process complex insurance text.
 3. Automate information retrieval using semantic search and vector storage.
 4. Enhance retrieval quality with re-ranking techniques.
 5. Design a modular, scalable, and customizable architecture.
 6. Support natural language queries and contextually accurate responses.
 7. Demonstrate an end-to-end pipeline from document ingestion to answer generation.
-

3. Problem Statement

The goal of the project is to build a robust generative search system capable of accurately answering questions from a wide range of insurance policy documents. These documents contain legal, structured, and domain-specific language that users typically struggle to interpret. Our solution should bridge this gap using modern LLM technologies.

4. Implementation

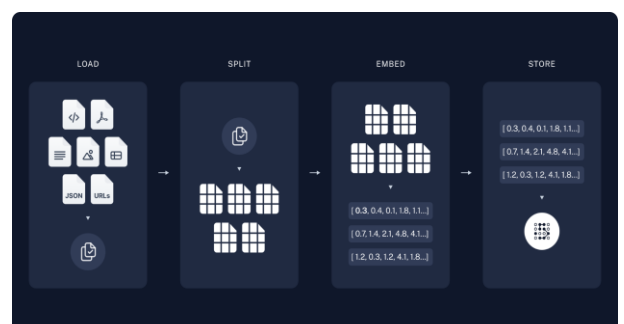
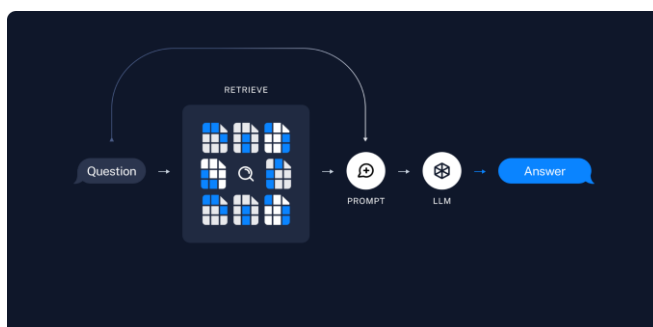
1. **Environment Setup**
 - Set up Python environment with required packages: `langchain`, `chromadb`, `openai`, `pypdf`.
2. **PDF Loading and Preprocessing**
 - Used `PyPDFDirectoryLoader` to read documents from a local directory.
 - Cleaned and filtered raw text before chunking.
3. **Chunking the Text**
 - Applied `RecursiveCharacterTextSplitter` to divide documents into overlapping chunks (~500 characters) with 50-character overlaps.
4. **Embedding Generation**
 - Used `OpenAIEmbeddings` to convert text chunks into vector form for semantic search.

5. **Storing in ChromaDB**
 - Stored the embeddings in `ChromaDB`, enabling persistent vector search and metadata storage.
 6. **Retriever Setup**
 - Used `VectorStoreRetriever` from `LangChain` to fetch top-k similar document chunks for a given query.
 7. **Re-Ranking**
 - Integrated HuggingFace `BAAI/bge-reranker-base` cross-encoder for relevance-based re-ranking of retrieved chunks.
 8. **RAG Chain Execution**
 - Loaded RAG chain from LangChain Hub (`rlm/rag-prompt`) and combined it with the retriever and LLM for final answer generation.
-

5. System Architecture

Overall Design: The system is composed of the following layers:

1. **Document Ingestion Layer**
 - Uses `PyPDFDirectoryLoader` from `LangChain` to read PDF documents.
2. **Text Chunking Layer**
 - Applies `RecursiveCharacterTextSplitter` to break down large documents into meaningful, overlapping chunks for improved semantic processing.
3. **Embedding Generation Layer**
 - Uses `OpenAIEmbeddings` from `LangChain` to convert text chunks into numerical vector representations.
4. **Vector Store Layer**
 - Stores embeddings using `ChromaDB` and supports fast similarity search and persistence.
5. **Retriever Layer**
 - Retrieves relevant chunks using `VectorStoreRetriever`, supporting flexible and efficient information access.
6. **Re-Ranking Layer**
 - Utilizes HuggingFace cross-encoder model `BAAI/bge-reranker-base` to re-rank results and improve retrieval relevance.
7. **RAG Chain Layer**
 - Combines user query with top-ranked context chunks using `rlm/rag-prompt` from LangChain Hub to generate answers via LLM.



6. Challenges Faced

- **Document Complexity:** Parsing and chunking lengthy legal documents without losing context.
 - **Latency:** Embedding and re-ranking operations introduce latency.
 - **Relevance Drift:** Without re-ranking, semantic search sometimes returns tangentially related results.
 - **Cost Considerations:** Embeddings and LLM calls can be costly at scale.
-

7. Lessons Learned

1. LangChain simplifies integration of LLMs and retrieval systems through its modular architecture.
2. Proper chunking significantly impacts retrieval accuracy.
3. Using cross-encoders for re-ranking substantially boosts result relevance.
4. Embeddings form the core of semantic understanding—quality and model choice matter.
5. RAG chains provide a powerful abstraction for combining retrieval and generation in a single step.